

Санкт-Петербургский политехнический университет
Институт компьютерных наук и кибербезопасности
Высшая школа программной инженерии

Отчет по курсовой работе
по теме «Преобразование изображения и GIF-анимации в
цветной ASCII-арт на Python»

Студент гр. 5130904/30002
Кладковой М.Д.
Преподаватель
Молодяков С.А.

Санкт-Петербург
2025

Цель работы

Разработать кросс-платформенную консольную программу на языке Python, которая:

1. Загружает растровое изображение (JPEG, PNG) или GIF-анимацию.
2. Преобразует каждый кадр в текст-графику (ASCII-арт), учитывая яркость пикселя.
3. При включённом флаге `--color` окрашивает каждый символ в цвет, близкий к цвету исходного пикселя (ANSI true-color).

Использованные технологии и библиотеки

1. **Pillow** — чтение растровых форматов и GIF, изменение размера, работа с цветом.
2. **NumPy** — быстрые векторные операции над пиксельными массивами.
3. **argparse** — разбор и валидация аргументов командной строки.
4. **colorama** (опционально) — облегчение работы с ANSI-кодами.

Описание алгоритма

1. Парсинг аргументов

С помощью `argparse` программа принимает:

- **input** (путь к файлу JPG/PNG или GIF)
- **-w, --width** (ширина выходного ASCII-изображения в символах, default=80)
- **-c, --chars** (строка символов от «тёмного» к «светлому», default="@%#*+=-:. ")
- **--color** (булев флаг: включить ANSI true-color).

2. Подготовка кадра

- Загружается изображение (`Image.open`).
- Вычисляется пропорциональная высота:
$$\text{new_height} = \text{int}(\text{width} * (\text{orig_height} / \text{orig_width}) * 0.55)$$

— коэффициент 0.55 компенсирует отличие соотношения символов.
- Создаются два массива:
 - **gray_arr** — оттенки серого (`.convert('L')`) для расчёта яркости;
 - **color_arr** — RGB-массив (`.convert('RGB')`) для извлечения цвета символа.

3. Построение ASCII-массива

- Для каждого элемента `gray_arr[y, x]` рассчитывается индекс символа:
$$\text{idx} = \text{gray} * (\text{len}(\text{chars}) - 1) // 255$$
- По `idx` формируется двумерный массив `chars[y, x]` из заданного набора символов.

4. Рендеринг в консоль

- **Без цвета:** просто объединяем строки из `chars`.
- **С цветом** (`--color`): для каждого символа берём `r, g, b = color_arr[y, x]` и вставляем ANSI-escape-последовательность:
`\x1b[38;2;{r};{g};{b}m{ch}\x1b[0m`

- Перед выводом — очищаем консоль (`cls/clear`), чтобы при GIF-режиме кадры сменяли друг друга.

5. Обработка режимов

- **Статическое изображение:** один проход, вывод и сохранение `output.txt`.
- **GIF-анимация:** итерация `ImageSequence.Iterator`, вывод кадров с задержкой `duration` (мс из GIF).

6. Завершение работы

После окончания преобразования (или при досрочном прерывании) ждём нажатия любой клавиши + Enter для выхода.

Описание флагов и параметров

Флаг / Параметр	Описание
<code>input</code>	Путь к исходному файлу (JPEG, PNG или GIF).
<code>-w, --width <число></code>	Ширина ASCII-изображения в символах. Чем меньше значение, тем грубее детализация.
<code>-c, --chars <строка></code>	Набор символов в порядке от самого «тёмного» к самому «светлому». По умолчанию <code>@%#*+=-: . . .</code>
<code>--color</code>	Включить цветной вывод: каждый символ окрашивается в RGB-значение исходного пикселя.

Таблица использованных функций

Функция / Класс	Модуль	Параметры	Описание
<code>Image.open(path)</code>	Pillow	<code>path: str</code>	Открыть изображение (JPEG, PNG, GIF)
<code>img.resize((w,h))</code>	Pillow	<code>w,h: int</code>	Изменить размер с сохранением пропорций
<code>img.convert('L')</code>	Pillow	—	Перевести изображение в оттенки серого
<code>img.convert('RGB')</code>	Pillow	—	Перевести изображение в RGB для true-color
<code>np.array(...)</code>	NumPy	<code>PIL.Image</code>	Преобразовать изображение в массив пикселей
<code>(gray*(n-1))/255</code>	NumPy	<code>gray: ndarray, n: int</code>	Вычислить индекс символа в наборе <code>chars</code>
<code>ANSI-escape \x1b[38;2;R;G;Bm{ch} \x1b[0m</code>	—	<code>R,G,B: int, ch: str</code>	Окраска символа в true-color в терминале
<code>os.system('clear')/cls</code>	os	—	Очистить консоль для анимации
<code>ImageSequence.Iterator(gif)</code>	Pillow	<code>gif: Image</code>	Итерация по кадрам анимации GIF
<code>time.sleep(duration)</code>	time	<code>duration: float</code>	Задержка между кадрами GIF (в секундах)
<code>argparse.ArgumentParser()</code>	argparse	—	Конфигурирование и разбор аргументов командной строки
<code>colorama.init(autoreset=True)</code>	colorama	—	Инициализация ANSI-

Функция / Класс

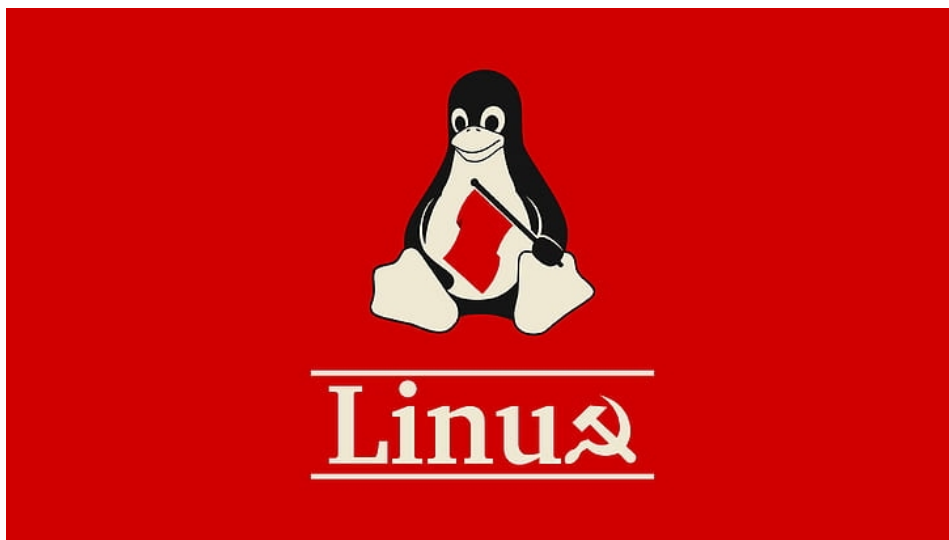
Модуль

Параметры

Описание
поддержки в Windows

Скриншоты работы программы

1. Исходное изображение



2. Резултат без цвета

[illegible]

3. Результат с цветом (- -color)

[illegible]

Фрагмент исходного кода

```
#!/usr/bin/env python3

import sys
import argparse
from PIL import Image, ImageSequence
import numpy as np
import time
import os

# Для true-color используем ANSI escape, colorama только для autoreast
try:
    from colorama import init as colorama_init
    colorama_init(autoreset=True)
    COLORAMA_OK = True
except ImportError:
    COLORAMA_OK = False

def parse_args():
    p = argparse.ArgumentParser(
        description="ASCII-арт (статичное или GIF) с опцией true-color"
    )
    p.add_argument("input", help="Путь к файлу (JPEG/PNG/GIF)")
    p.add_argument("-w", "--width", type=int, default=80,
        help="Ширина в символах")
    p.add_argument("-c", "--chars", default="@%#*+=-:. ",
        help="Символы от тёмного к светлому")
    p.add_argument("--color", action="store_true",
        help="Использовать true-color из исходного изображения")
    return p.parse_args()

def clear_console():
    os.system('cls' if os.name == 'nt' else 'clear')

def load_and_prepare(img, new_width):
    w, h = img.size
    ratio = h / w
```

```

new_h = int(new_width * ratio * 0.55)
resized = img.resize((new_width, new_h))
gray = np.array(resized.convert('L'), dtype=int)
color = np.array(resized.convert('RGB'), dtype=int)
return gray, color

```

```

def map_pixels_to_ascii(gray, charset):
    levels = len(charset)
    idx = (gray * (levels - 1)) // 255
    chars = np.array([charset[i] for i in idx.ravel()]).reshape(idx.shape)
    return chars, idx

```

```

def render_ascii(chars, idx, color_arr=None, use_color=False):
    lines = []
    h, w = chars.shape
    for y in range(h):
        line = ''
        for x in range(w):
            ch = chars[y, x]
            if use_color and color_arr is not None:
                r, g, b = color_arr[y, x]
                line += f"\x1b[38;2;{r};{g};{b}m{ch}\x1b[0m"
            else:
                line += ch
        lines.append(line)
    return '\n'.join(lines)

```

```

def process_static_image(path, args):
    img = Image.open(path)
    gray, color_arr = load_and_prepare(img, args.width)
    chars, idx = map_pixels_to_ascii(gray, args.chars)
    art = render_ascii(chars, idx, color_arr, args.color)
    clear_console()
    print(art)
    with open('output.txt', 'w') as f:
        f.write(art)

```

```
def process_gif(path, args):
    gif = Image.open(path)
    duration = gif.info.get('duration', 100) / 1000.0
    for frame in ImageSequence.Iterator(gif):
        gray, color_arr = load_and_prepare(frame, args.width)
        chars, idx = map_pixels_to_ascii(gray, args.chars)
        art = render_ascii(chars, idx, color_arr, args.color)
        clear_console()
        print(art)
        time.sleep(duration)

def main():
    args = parse_args()
    if args.input.lower().endswith('.gif'):
        process_gif(args.input, args)
    else:
        process_static_image(args.input, args)
    input("\nНажмите любую клавишу и Enter для выхода...")

if __name__ == "__main__":
    main()
```